MIT 8103 Advanced Database Systems: Portfolio Summary
Case study name: Pear-C Goods Distribution System
Name: EDOZIEUNO CHINENYE
Student ID: 2026/A/MIT/0695
Repository: https://github.com/rita-edozie/Edozieuno_Chinenye_301927088.git

Case study

The Pear-C Goods Distribution case study models a regional Fast-Moving Consumer Goods (FMCG) distribution network. The operational scope covers categories, products, varieties, multi-warehouse stock entries, active and inactive distributors, and distributor orders with associated order items. It evaluates both relational data integrity and modern NoSQL event tracking for supply chain operations within a single regional center, explicitly excluding retail point-of-sale transactions, payroll processing, and third-party logistics vehicle tracking.

Portfolio 1: Database Design and Modelling

This portfolio established the foundational relational schema and declarative integrity constraints for the Pear-C database in PostgreSQL (Edozieuno_Chinenye_301927088_P1_README.md).
What was done: Designed entity-relationship models and implemented table definitions with primary keys, foreign keys, and check constraints (Edozieuno_Chinenye_301927088_P1_Constraints.sql). Negative test cases were created to test constraint enforcement under explicit transaction blocks (BEGIN and ROLLBACK).
Verified result: All negative test cases executed successfully. PostgreSQL intercepted and rejected attempts to insert products with non-existent category IDs, insert products with NULL names, reference non-existent order or distributor IDs in foreign keys, and submit non-positive quantities. Wrapped in explicit transactions, all constraint violation errors triggered rollbacks preserving data integrity. Positive sample data insertion scripts populated valid categories, products, distributors, and orders without errors.
Meaning: While declarative constraints effectively maintain atomic data validity and referential integrity, complex business rules—such as validating that order quantities do not exceed available warehouse stock (Rule 3), enforcing distributor deletion restrictions based on active order statuses (Rule 6), and ensuring a minimum of one order item per distributor order (Rule 2)—require supplementary database triggers or application-level validation logic (Elmasri & Navathe, 2015).

Portfolio 2: Query Processing and Optimisation

This portfolio analyzed core relational queries and benchmarked performance modifications on a synthetic history table (p2_benchmark.distributor_order_history) containing 100,000 rows.
What was done: Implemented and tested queries filtering by distributor ID and order date ranges, evaluated execution plans using EXPLAIN (ANALYZE, BUFFERS), and created a composite index (idx_dist_order_hist_dist_date) to measure performance shifts.
Verified result: The query planner continued executing a Sequential Scan (Seq Scan) after creating the index because the estimated cost threshold for fetching that percentage of rows did not justify an index lookup over a full table read.
Before and After Index Benchmark

Phase	Scan Type	Three Execution Times Median Time	Shared Buffer Hits (Run 3)
Before Index	Seq Scan	18.552 ms, 20.076 ms, 13.111 ms 18.552 ms	shared hit=1274
After Index	Seq Scan	18.210 ms, 15.030 ms, 8.111 ms 15.030 ms	shared hit=1274

Measurement Limits & Units: Times are measured in milliseconds (ms). Micro-benchmarks exhibit inherent timing variance due to OS cache states, shared local resource utilization, and CPU scheduler jitter rather than structural

query alterations. Furthermore, maintaining indexes introduces real storage and write overhead during INSERT, UPDATE, and DELETE operations, highlighting that indexes are selective tools rather than absolute guarantees of improved performance (PostgreSQL Global Development Group, 2026).

Portfolio 3: Transactions and Concurrency

This portfolio examined transaction management, ACID properties, and concurrency control in PostgreSQL (Edozieuno_Chinenye_301927088_P3_README.md).
What was done: Executed multi-statement transaction blocks (BEGIN, COMMIT, ROLLBACK) to update distributor statuses and handle constraint violations.
Verified result & State Distinctions:
Earlier Database State: Distributors 101 and 103 held their initial pre-transaction operational statuses.
Later Transaction Changes: Transaction 1 successfully updated and committed distributor 103's status to 'Active'. Subsequent runs safely affected 0 rows. Transaction 2 successfully updated distributor 101 to 'Inactive' inside a transaction block, but a deliberate subsequent statement attempting to update distributor 102's status to 'Pending' violated table constraints. The entire transaction block was rolled back via ROLLBACK. Post-check queries confirmed that distributor 103 remained permanently 'Active' (from Transaction 1's commit), while distributor 101 remained 'Active' (demonstrating that Transaction 2's uncommitted changes were successfully discarded).
Meaning: Database isolation and transaction controls protect against conflicting modifications, but cross-table business rules require robust application or trigger validation.

Portfolio 4: NoSQL and Advanced Data Models

This portfolio explored document-oriented data storage for supply chain event logs using MongoDB (Edozieuno_Chinenye_301927088_P4_README.md).
What was done: Configured MongoDB version 8.0.34 locally inside an Ubuntu Codespace (/workspaces/mongodb-data). Created a supply_chain_events collection and inserted four fictional documents (EVT-1001, EVT-1008, EVT-2001, and EVT-2002) using mongosh via a single insertMany block. Performed nested-field queries filtering for Distributor 101 ({ "sourceLocation.distributorId": 101 }).
Verified result: The insertMany block yielded successful acknowledgment with inserted IDs, and db.supply_chain_events.countDocuments() returned 4. Nested-field queries successfully retrieved matching events (such as EVT-1001 for Lamdan Retailers) sorted chronologically. The core PostgreSQL relational database remained entirely unchanged and operational.
Meaning: Document databases provide high flexibility for semi-structured log data without requiring rigid relational schemas, perfectly complementing relational databases.

Portfolio 5: Distributed and Cloud Database Exercise

This portfolio investigated distributed database architectures, cloud replication strategies, and fault tolerance (Edozieuno_Chinenye_301927088_P5_README.md).
What was done: Evaluated replication topologies and failure recovery for MongoDB.
Verified Result & Proposed Design:
Proposed Distributed Design: To provide high availability and fault tolerance for the event log, a three-member replica set is proposed (mongo-node-1 in Zone A as Primary, mongo-node-2 and mongo-node-3 in Zones B and C as Secondaries). Utilizing MongoDB Write Concern with w: "majority" ensures writes are acknowledged only after successful replication to a majority of nodes.
Tests Performed vs. Not Performed: Theoretical failure-mode analysis and configuration modeling were performed. Live multi-node cluster topologies, active replication status verification, multi-region network partition stress tests,

and automated failover latency load tests were not performed due to the standalone local Ubuntu Codespace environment constraints. If a primary node fails, remaining nodes trigger an election following MongoDB Replica Set Election guidelines; if a majority of voting nodes become unreachable, the replica set becomes read-only (MongoDB, Inc., 2026).
Meaning: Distributed cloud configurations trade minor increases in write latency for high availability, while core relational rules remain anchored under PostgreSQL.

Evidence

Evidence of completed work, SQL scripts, JSON documents, and README documentation is stored in the project repository:
Edozieuno_Chinenye_301927088_P1_README.md, Edozieuno_Chinenye_301927088_P1_Constraints.sql, Edozieuno_Chinenye_301927088_P1_Design.md, Edozieuno_Chinenye_301927088_P1_ERDiagram.md, Edozieuno_Chinenye_301927088_P1_SampleData.sql, Edozieuno_Chinenye_301927088_P1_Schema.sql, screenshots
Edozieuno_Chinenye_301927088_P2_README.md, Edozieuno_Chinenye_301927088_P2_Optimisation.sql, Edozieuno_Chinenye_301927088_P2_Queries.sql, screenshots
Edozieuno_Chinenye_301927088_P3_README.md, Edozieuno_Chinenye_301927088_P3_SessionA.sql, Edozieuno_Chinenye_301927088_P3_SessionB.sql, Edozieuno_Chinenye_301927088_P3_Transactions.sql, screenshots
Edozieuno_Chinenye_301927088_P4_README.md, Edozieuno_Chinenye_301927088_P4_MongoDBSource.list, Edozieuno_Chinenye_301927088_P4_Setup.md, Edozieuno_Chinenye_301927088_P4_MongoDB.js, screenshots
Edozieuno_Chinenye_301927088_P5_README.md, screenshots

Individual reflection

Design choice: I decided to use a Fast-Moving Consumer Goods system because I would like more insight into how to transform the legacy system into a digital system.
Difficulty: I found navigating the MongoDB database more challenging because I do not have prior knowledge of it. I also found the optimisation and indexing part quite difficult as well, but I was able to navigate my way around it.
Action: I intend to research more and seek guidance on how to navigate the database and its application.
Lesson: I was able to navigate GitHub by creating repositories, writing queries, inputting commands that generated diagrams, and taking screenshots as evidence of work done.

References

Elmasri, R., & Navathe, S. B. (2015). Fundamentals of Database Systems (7th ed.). Pearson.
PostgreSQL Global Development Group. (2026). PostgreSQL Documentation: Indices, Transaction Isolation, and Constraints. https://www.postgresql.org/docs/
MongoDB, Inc. (2026). MongoDB Manual: Replica Sets, Write Concerns, and mongosh Operations. https://www.mongodb.com/docs/

AI-use declaration

Tools: Google Gemini
Help received: Helped with the writing of queries, stating reasons that helped me understand each step better.
My verification: I personally ran the queries on GitHub, created my portfolios, formulated every figure and name related to my case study, documented all evidence, and saved it in my repository.